# CAFA5 Protein Function Prediction: Data Preparation and Exploration

**Course:** BioE 242 Final Project
**Team:** Benson Li, Yu-Shan Fu

---

## Project Background

**CAFA (Critical Assessment of Functional Annotation)** is an international benchmarking challenge for protein function prediction.
Given a protein sequence, the task is to predict its **Gene Ontology (GO)** labels — which biological functions the protein performs.

GO labels are organized into three namespaces:

| Abbreviation | Full Name | Description |
|---|---|---|
| BPO | Biological Process | The biological process the protein participates in (e.g., DNA repair) |
| MFO | Molecular Function | The molecular activity of the protein itself (e.g., kinase activity) |
| CCO | Cellular Component | Where in the cell the protein is located (e.g., nucleus) |

This is a **multi-label classification** problem: each protein can have multiple GO term annotations simultaneously.

---

## Dataset

CAFA5 training set (Kaggle) contains:

- `train_sequences.fasta` — 142,246 protein sequences
- `train_terms.tsv` — 5,363,863 (protein, GO term, namespace) triples
- `train_taxonomy.tsv` — taxonomy information (species IDs)

---

## Feature Engineering Strategies Covered in This Notebook

| Strategy | Method | Feature Dim | Characteristics |
|---|---|---|---|
| K-mer frequency | Count sliding 2-mer windows | 400 | Fast, interpretable baseline |

| Strategy | Method | Feature Dim | Characteristics |
|---|---|---|---|
| ESM2 embeddings | Protein language model | 320 | Rich semantics, pre-trained knowledge |
| AlphaFold structural | 3D structure → graph data | variable | Spatial interaction information |

## Notebook Structure

1. Environment setup
2. Raw data loading and initial exploration
3. Data cleaning (remove invalid sequences)
4. GO label alignment and multi-label encoding
5. Feature engineering 1: K-mer frequency vectors
6. Feature engineering 2: ESM2 protein language model embeddings
7. Feature engineering 3: AlphaFold structural features (graph cache walkthrough)
8. Summary and next steps

## 1. Environment Setup

Install required packages and define all file paths.

**Design principle:** All paths are defined in one place at the top of the notebook. When switching environments (local machine / Savio HPC / Colab), only this section needs to change.

In [1]:
```
import sys
!{sys.executable} -m pip install numpy pandas biopython scikit-learn scip
```

In [2]:
```
import os
import pickle
from itertools import product

import numpy as np
import pandas as pd
from Bio import SeqIO
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MultiLabelBinarizer, StandardScaler
from scipy.sparse import csr_matrix, save_npz
import matplotlib.pyplot as plt

BASE_DIR = "/global/scratch/users/bensonli/cafa5"

TRAIN_DIR = os.path.join(BASE_DIR, "Train")

train_fasta_path    = os.path.join(TRAIN_DIR, "train_sequences.fasta")
train_terms_path    = os.path.join(TRAIN_DIR, "train_terms.tsv")
train_taxonomy_path = os.path.join(TRAIN_DIR, "train_taxonomy.tsv")
```

# 2. Raw Data Loading and Initial Exploration

## Why this step?

Before any cleaning or feature engineering, understanding the data at a high level helps us:

- Confirm the data was downloaded correctly (row counts match the official documentation)
- Detect potential issues early (null values, duplicate IDs, malformed sequences)
- Build intuition about the dataset before designing features

## What we load

| File | Format | Content |
|---|---|---|
| `train_sequences.fasta` | FASTA | Protein ID → amino acid sequence |
| `train_terms.tsv` | TSV | (EntryID, GO term, namespace) triples |
| `train_taxonomy.tsv` | TSV | (EntryID, taxonomy ID) pairs |

In [3]:
```
# Load protein sequences from FASTA
sequences_dict     = {}
duplicate_id_count = 0
missing_id_count   = 0
empty_seq_count    = 0

for record in SeqIO.parse(train_fasta_path, "fasta"):
    seq_id = record.id
    seq    = str(record.seq).strip()

    if not seq_id:
        missing_id_count += 1
        continue
    if seq_id in sequences_dict:
        duplicate_id_count += 1
    if len(seq) == 0:
        empty_seq_count += 1

    sequences_dict[seq_id] = seq

print(f"Sequences loaded:      {len(sequences_dict):>8,}")
print(f"Duplicate IDs:         {duplicate_id_count:>8,}")
print(f"Missing IDs:           {missing_id_count:>8,}")
print(f"Empty sequences:       {empty_seq_count:>8,}")

# Load GO label table and taxonomy
terms    = pd.read_csv(train_terms_path,    sep="\t")
taxonomy = pd.read_csv(train_taxonomy_path, sep="\t")

print(f"\ntrain_terms shape:    {terms.shape}")
print(f"train_taxonomy shape:  {taxonomy.shape}")
terms.head()
```

```
Sequences loaded:       142,246
Duplicate IDs:                0
Missing IDs:                  0
Empty sequences:              0

train_terms shape:    (5363863, 3)
train_taxonomy shape:  (142246, 2)
```

Out[3]:

| | EntryID | term | aspect |
|---|---|---|---|
| 0 | A0A009IHW8 | GO:0008152 | BPO |
| 1 | A0A009IHW8 | GO:0034655 | BPO |
| 2 | A0A009IHW8 | GO:0072523 | BPO |
| 3 | A0A009IHW8 | GO:0044270 | BPO |
| 4 | A0A009IHW8 | GO:0006753 | BPO |

In [4]:

```python
# Missing value check
# Purpose: ensure downstream alignment and label encoding will not silent
print("Missing values in train_terms:")
print(terms.isnull().sum())
print()
print("Missing values in train_taxonomy:")
print(taxonomy.isnull().sum())
```

```
Missing values in train_terms:
EntryID    0
term       0
aspect     0
dtype: int64

Missing values in train_taxonomy:
EntryID       0
taxonomyID    0
dtype: int64
```

In [5]:

```python
# Drop rows where EntryID or term is missing (defensive cleanup)
terms = terms.dropna(subset=["EntryID", "term"]).copy()
print(f"train_terms shape after null removal: {terms.shape}")
```

```
train_terms shape after null removal: (5363863, 3)
```

# 3. Data Cleaning: Remove Invalid Sequences

## Why this step?

Protein sequences should only contain the 20 standard amino acids ( `ACDEFGHIKLMNPQRSTVWY` ).
The CAFA5 dataset contains sequences with non-standard characters:

| Symbol | Meaning | Reason to remove |
|---|---|---|
| `X` | Unknown amino acid | Cannot produce meaningful k-mers or embeddings |
| `U` | Selenocysteine | Rare, standard models are not designed for it |

| Symbol | Meaning | Reason to remove |
|---|---|---|
| `O` | Pyrrolysine | Rare, same issue |
| `B`, `Z` | Ambiguous residue codes | Represent sequencing uncertainty |

**Keeping these sequences would cause problems:**

- K-mer vocabulary becomes incomplete
- ESM2 tokenizer may reject or silently truncate them
- AlphaFold structures for these proteins typically do not exist in the database

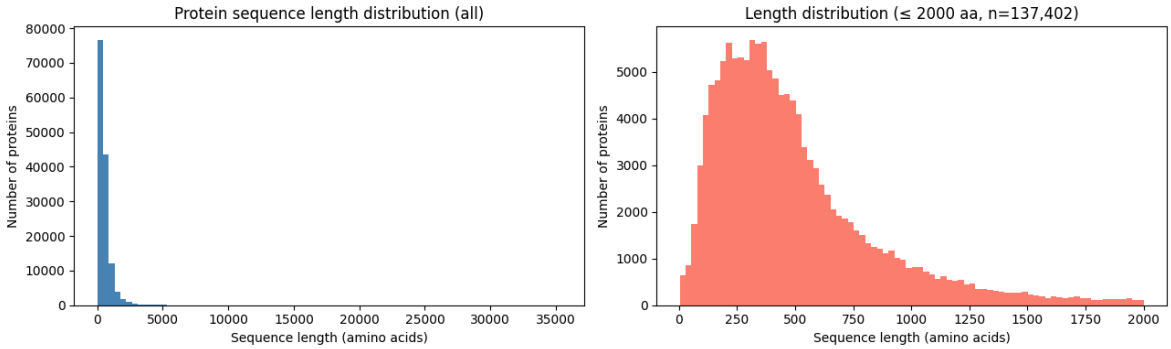
**Strategy: remove them and log the removed IDs for auditability.**

In [6]:
```
VALID_AAS = set("ACDEFGHIKLMNPQRSTVWY")

clean_sequences_dict = {}
removed_empty   = []
removed_invalid = []   # list of (protein_id, set_of_bad_chars)

for seq_id, seq in sequences_dict.items():
    seq = seq.strip()
    if len(seq) == 0:
        removed_empty.append(seq_id)
        continue
    bad_chars = set(seq) - VALID_AAS
    if bad_chars:
        removed_invalid.append((seq_id, bad_chars))
        continue
    clean_sequences_dict[seq_id] = seq

print(f"Removed empty sequences:              {len(removed_empty):>6,}")
print(f"Removed sequences with invalid chars: {len(removed_invalid):>6,}"
print(f"Remaining clean sequences:            {len(clean_sequences_dict):

# Inspect a few removed examples
print("\nSample removed sequences (first 10):")
for pid, bad in removed_invalid[:10]:
    print(f"  {pid}: contains {bad}")

sequences_dict = clean_sequences_dict
```

```
Removed empty sequences:                   0
Removed sequences with invalid chars:  1,677
Remaining clean sequences:           140,569

Sample removed sequences (first 10):
  Q86KP5: contains {'U'}
  O93661: contains {'O'}
  Q9JLT4: contains {'U'}
  Q8FDI4: contains {'X'}
  P11145: contains {'X'}
  P86782: contains {'X'}
  P18173: contains {'U'}
  P80473: contains {'X'}
  P80803: contains {'X'}
  P84712: contains {'X'}
```

In [7]:
```
# Sequence length statistics and distribution
lengths = [len(seq) for seq in sequences_dict.values()]

print(f"Number of proteins:   {len(sequences_dict):>8,}")
print(f"Mean length:          {np.mean(lengths):>10.1f}")
print(f"Median length:        {np.median(lengths):>10.1f}")
print(f"Min length:           {np.min(lengths):>10,}")
print(f"Max length:           {np.max(lengths):>10,}")
print(f"Std dev:              {np.std(lengths):>10.1f}")

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

# Full distribution
axes[0].hist(lengths, bins=80, color="steelblue", edgecolor="none")
axes[0].set_xlabel("Sequence length (amino acids)")
axes[0].set_ylabel("Number of proteins")
axes[0].set_title("Protein sequence length distribution (all)")

# Truncated at 2000 to show the main body
clipped = [l for l in lengths if l <= 2000]
axes[1].hist(clipped, bins=80, color="salmon", edgecolor="none")
axes[1].set_xlabel("Sequence length (amino acids)")
axes[1].set_ylabel("Number of proteins")
axes[1].set_title(f"Length distribution (≤ 2000 aa, n={len(clipped):,})")

plt.tight_layout()
plt.show()

long_seqs = sum(1 for l in lengths if l > 2000)
print(f"\nProteins with length > 2000: {long_seqs:,} ({100*long_seqs/len(
#Note: ESM2 truncates sequences at 1022 tokens. Longer sequences lose C-t
#Graph-based methods (AlphaFold pipeline) handle arbitrary-length protein
```

```
Number of proteins:    140,569
Mean length:               553.7
Median length:             412.0
Min length:                    3
Max length:               35,375
Std dev:                   639.7
```




```
Proteins with length > 2000: 3,167 (2.3%)
```

# 4. GO Label Alignment and Multi-label Encoding

## Why this step?

Sequence data (FASTA) and label data (TSV) are stored separately.

We need to ensure:

1. Every protein in our training set has both a sequence **and** at least one GO label
2. The order of sequences and labels is perfectly aligned (same index = same protein)

After removing 1,677 sequences with non-standard amino acids, the `terms` table still contains labels for those proteins.
Alignment explicitly resolves this mismatch.

## Multi-label Binarization

Each protein has **multiple** GO terms.
`MultiLabelBinarizer` transforms the list of GO terms into a binary matrix **Y** of shape **(N proteins × M GO terms)**,
where `Y[i, j] = 1` means protein *i* is annotated with GO term *j*.

This matrix is **very sparse**: each protein has on average only a few dozen positive labels out of 31,000+ total GO terms.

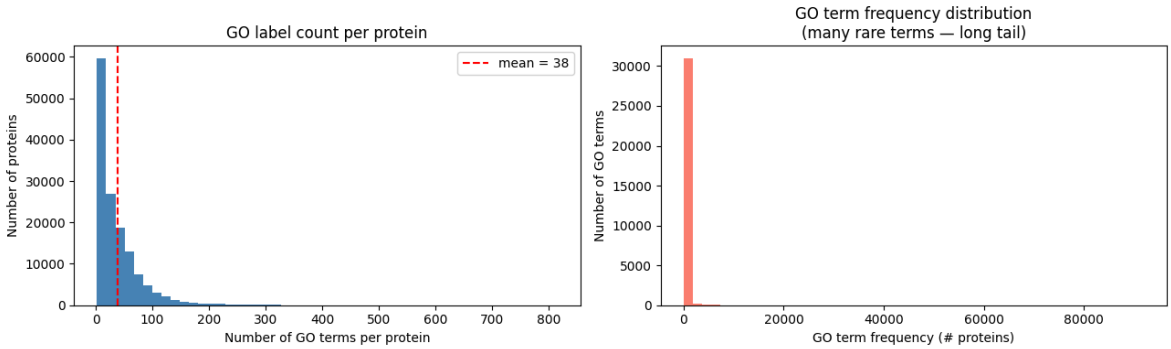
In [8]:
```
# Build label dictionary: EntryID -> list of GO terms
labels_dict = terms.groupby("EntryID")["term"].apply(list)

sequence_ids = set(sequences_dict.keys())
label_ids    = set(labels_dict.index)

print(f"Proteins with sequence only (no labels): {len(sequence_ids - labe
print(f"Proteins with labels only (no sequence): {len(label_ids - sequenc
print(f"Proteins with both sequence and labels:  {len(sequence_ids & labe

# Align: keep only proteins that have both sequence and labels, preservin
protein_ids = []
sequences   = []
all_labels  = []

for pid, seq in sequences_dict.items():
    if pid in labels_dict:
        protein_ids.append(pid)
        sequences.append(seq)
        all_labels.append(labels_dict[pid])

print(f"\nFinal aligned proteins: {len(sequences):,}")
```

```
Proteins with sequence only (no labels):      0  (excluded)
Proteins with labels only (no sequence):  1,677  (from removed invalid seq
s)
Proteins with both sequence and labels:  140,569

Final aligned proteins: 140,569
```

In [9]:
```
# Multi-label binarization
mlb = MultiLabelBinarizer()
Y   = mlb.fit_transform(all_labels).astype(np.int8)

print(f"Y shape:                        {Y.shape}")
print(f"Number of GO terms:             {len(mlb.classes_):,}")
print(f"Sparsity (fraction of zeros):   {1 - Y.sum() / Y.size:.4f}")
```

```
# Labels per protein distribution
labels_per_protein = Y.sum(axis=1)
label_counts       = Y.sum(axis=0)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].hist(labels_per_protein, bins=50, color="steelblue", edgecolor="n
axes[0].set_xlabel("Number of GO terms per protein")
axes[0].set_ylabel("Number of proteins")
axes[0].set_title("GO label count per protein")
axes[0].axvline(np.mean(labels_per_protein), color="red", linestyle="--",
                label=f"mean = {np.mean(labels_per_protein):.0f}")
axes[0].legend()

axes[1].hist(label_counts, bins=50, color="salmon", edgecolor="none")
axes[1].set_xlabel("GO term frequency (# proteins)")
axes[1].set_ylabel("Number of GO terms")
axes[1].set_title("GO term frequency distribution\n(many rare terms — lon

plt.tight_layout()
plt.show()

print(f"\nMean labels per protein:            {np.mean(labels_per_protein
print(f"GO terms with frequency >= 50:      {(label_counts >= 50).sum():,
print(f"GO terms with frequency <  10:      {(label_counts < 10).sum():,}
```

```
Y shape:                        (140569, 31454)
Number of GO terms:             31,454
Sparsity (fraction of zeros):   0.9988
```

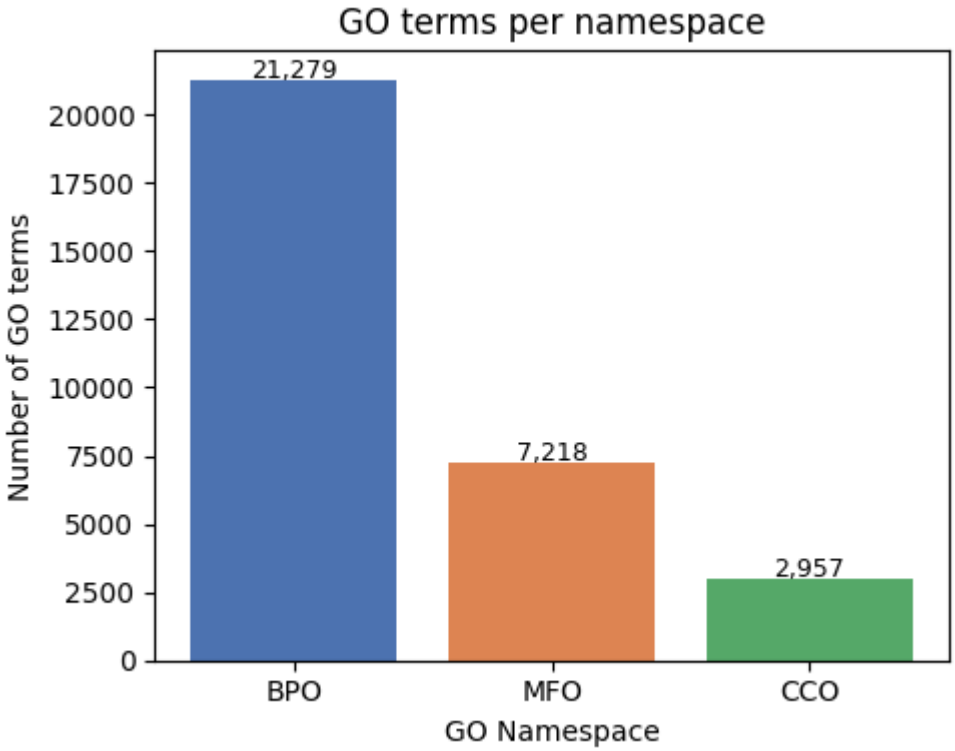



```
Mean labels per protein:            37.8
GO terms with frequency >= 50:      6,398
GO terms with frequency <  10:      16,588
```

In [10]:
```
# GO term breakdown by namespace (BPO / MFO / CCO)
terms_indexed = terms[terms["term"].isin(mlb.classes_)]
aspect_dict   = terms_indexed.drop_duplicates("term").set_index("term")["

aspect_series = pd.Series([aspect_dict.get(go, "Unknown") for go in mlb.c
aspect_counts = aspect_series.value_counts()

print("GO term distribution by namespace:")
print(aspect_counts.to_string())

fig, ax = plt.subplots(figsize=(5, 4))
colors = ["#4C72B0", "#DD8452", "#55A868"]
bars = ax.bar(aspect_counts.index, aspect_counts.values, color=colors)
ax.set_xlabel("GO Namespace")
ax.set_ylabel("Number of GO terms")
ax.set_title("GO terms per namespace")
for bar, v in zip(bars, aspect_counts.values):
```

```
    ax.text(bar.get_x() + bar.get_width()/2, v + 80, f"{v:,}", ha="center
plt.tight_layout()
plt.show()
```

```
GO term distribution by namespace:
BPO    21279
MFO     7218
CCO     2957
```




# 5. Feature Engineering 1: K-mer Frequency Vectors

## What is a k-mer?

Scan a protein sequence with a sliding window of length *k* and count how often each possible k-mer appears.
Normalize by the total number of windows to get **relative frequencies** independent of sequence length.

Example for sequence `ACDEF` with k=2:
Windows: `AC`, `CD`, `DE`, `EF` → frequency of each 2-mer divided by 4

## Why use k-mer features?

| Advantage | Explanation |
|---|---|
| Speed | No GPU needed; computed in minutes for 140k proteins |
| Interpretable | Each feature corresponds to a specific amino acid pair |
| Baseline | Provides a fast baseline to compare against deep learning approaches |
| Captures local composition | Adjacent amino acid preferences are informative for function |

## Why k=2?

| k | Vocabulary size | Notes |
|---|---|---|
| 1 | 20 | Only amino acid composition — misses local context |
| **2** | **400** | Captures adjacent AA interactions — good balance |
| 3 | 8,000 | Larger but sparser; diminishing returns |
| 4+ | 160,000+ | Increasingly sparse; mostly zeros for short sequences |

**We use k=2 as the standard baseline feature.**

## Why normalize to frequencies (not raw counts)?

Raw counts are biased by sequence length.
A 1000 aa protein will have ~10× more total k-mers than a 100 aa protein,
even if they have identical amino acid composition.
Dividing by the total number of windows removes this length bias.

In [11]:
```
# Build k=2 vocabulary (all 20^2 = 400 possible 2-mers)
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
K           = 2

kmer_vocab  = [''.join(p) for p in product(AMINO_ACIDS, repeat=K)]
kmer_to_idx = {kmer: i for i, kmer in enumerate(kmer_vocab)}

print(f"k = {K}")
print(f"Vocabulary size: {len(kmer_vocab)}  (= 20^{K})")
print(f"First 10 k-mers: {kmer_vocab[:10]}")
```

```
k = 2
Vocabulary size: 400  (= 20^2)
First 10 k-mers: ['AA', 'AC', 'AD', 'AE', 'AF', 'AG', 'AH', 'AI', 'AK', 'A
L']
```

In [12]:
```
def seq_to_kmer_vector(seq: str, kmer_to_idx: dict, k: int = 2) -> np.nda
    """
    Convert a protein sequence to a normalized k-mer frequency vector.

    Normalization: divide raw counts by (len(seq) - k + 1) so that the
    vector represents relative frequencies rather than absolute counts.
    This makes features comparable across proteins of different lengths.
    """
    vec   = np.zeros(len(kmer_to_idx), dtype=np.float32)
    total = len(seq) - k + 1

    if total <= 0:
        return vec   # sequence too short for any k-mer

    for i in range(total):
        kmer = seq[i:i+k]
        if kmer in kmer_to_idx:
            vec[kmer_to_idx[kmer]] += 1.0

    vec /= total
```

```
    return vec

# Compute k-mer feature matrix for all sequences
print("Computing k-mer feature matrix (this takes ~1-2 minutes)...")
X_kmer = np.array(
    [seq_to_kmer_vector(seq, kmer_to_idx, k=K) for seq in sequences],
    dtype=np.float32
)

print(f"X_kmer shape: {X_kmer.shape}")
print(f"Feature value range: [{X_kmer.min():.4f}, {X_kmer.max():.4f}]")
```

```
Computing k-mer feature matrix (this takes ~1-2 minutes)...
X_kmer shape: (140569, 400)
Feature value range: [0.0000, 0.5077]
```

In [13]:

```
# Train / validation split (80/20)
indices = np.arange(len(X_kmer))
train_idx, val_idx = train_test_split(indices, test_size=0.2, random_stat

X_kmer_train = X_kmer[train_idx]
X_kmer_val   = X_kmer[val_idx]
Y_train       = Y[train_idx]
Y_val         = Y[val_idx]

print(f"X_kmer_train: {X_kmer_train.shape}")
print(f"X_kmer_val:   {X_kmer_val.shape}")
print(f"Y_train:      {Y_train.shape}")
print(f"Y_val:        {Y_val.shape}")
```

```
X_kmer_train: (112455, 400)
X_kmer_val:   (28114, 400)
Y_train:      (112455, 31454)
Y_val:        (28114, 31454)
```

In [26]:

```
# Feature standardization (zero mean, unit variance)
scaler       = StandardScaler()
X_kmer_train = scaler.fit_transform(X_kmer_train).astype(np.float32)
X_kmer_val   = scaler.transform(X_kmer_val).astype(np.float32)

print(f"X_kmer_train  mean :  {X_kmer_train.mean():.4f}")
print(f"X_kmer_train  std  :  {X_kmer_train.std():.4f}")
```

```
X_kmer_train  mean :  -0.0000
X_kmer_train  std  :  1.0000
```

In [15]:

```
# Save k-mer processed outputs
OUTPUT_KMER = "data_processed_kmer"
os.makedirs(OUTPUT_KMER, exist_ok=True)

np.save(os.path.join(OUTPUT_KMER, "X_train.npy"),   X_kmer_train)
np.save(os.path.join(OUTPUT_KMER, "X_val.npy"),     X_kmer_val)
save_npz(os.path.join(OUTPUT_KMER, "Y_train.npz"),  csr_matrix(Y_train))
save_npz(os.path.join(OUTPUT_KMER, "Y_val.npz"),    csr_matrix(Y_val))
np.save(os.path.join(OUTPUT_KMER, "train_idx.npy"), train_idx)
np.save(os.path.join(OUTPUT_KMER, "val_idx.npy"),   val_idx)

# Save metadata: scaler, vocabulary, protein IDs, and MultiLabelBinarizer
with open(os.path.join(OUTPUT_KMER, "meta.pkl"), "wb") as f:
    pickle.dump({
        "scaler":      scaler,
```

```
        "k":            K,
        "kmer_vocab":   kmer_vocab,
        "kmer_to_idx":  kmer_to_idx,
        "protein_ids":  protein_ids,
        "mlb":          mlb,
    }, f)

print(f"K-mer features saved to: {OUTPUT_KMER}/")
print(os.listdir(OUTPUT_KMER))
```

```
K-mer features saved to: data_processed_kmer/
['X_train.npy', 'X_val.npy', 'Y_train.npz', 'Y_val.npz', 'meta.pkl', 'trai
n_idx.npy', 'val_idx.npy']
```

# 6. Feature Engineering 2: ESM2 Protein Language Model Embeddings

## What is ESM2?

**ESM2 (Evolutionary Scale Modeling 2)** is a protein language model released by Meta AI,
pre-trained on hundreds of millions of protein sequences using self-supervised learning.
It learns to encode:

- Evolutionary patterns and conservation
- Local and global sequence context via self-attention
- Implicit structural and functional signals

## Why ESM2 outperforms K-mer features?

| Property | K-mer | ESM2 |
|---|---|---|
| Long-range dependencies | No (only adjacent AAs) | Yes (full-sequence attention) |
| Semantic understanding | No | Yes (pre-trained knowledge) |
| Transfer learning | No | Yes |
| Compute cost | Low | High (~2 hours on CPU) |

## Which ESM2 model do we use?

We use `facebook/esm2_t6_8M_UR50D` — the smallest ESM2 variant:

- 6 transformer layers
- 8M parameters
- Embedding dimension = **320**
- Runs on CPU (~2 hours for 140k sequences)

## How we extract the embedding

Feed the full sequence into ESM2 and take the **mean of the last hidden states** across all residue tokens (mean pooling).

This produces one fixed-size vector per protein regardless of sequence length.

In [16]:
```
import torch
from transformers import AutoTokenizer, AutoModel

MODEL_NAME = "facebook/esm2_t6_8M_UR50D"
device     = torch.device("cuda" if torch.cuda.is_available() else "cpu")

tokenizer  = AutoTokenizer.from_pretrained(MODEL_NAME)
esm_model  = AutoModel.from_pretrained(MODEL_NAME).to(device)
esm_model.eval()

print(f"Device:           {device}")
print(f"Model:            {MODEL_NAME}")
print(f"Embedding dim:    320")
```

```
/global/home/users/bensonli/venvs/cafa5-notebook/lib/python3.11/site-packa
ges/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyt
er and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_in
stall.html
  from .autonotebook import tqdm as notebook_tqdm
/global/home/users/bensonli/venvs/cafa5-notebook/lib/python3.11/site-packa
ges/torch/cuda/__init__.py:180: UserWarning: CUDA initialization: The NVID
IA driver on your system is too old (found version 12030). Please update y
our GPU driver by downloading and installing a new version from the URL: h
ttp://www.nvidia.com/Download/index.aspx Alternatively, go to: https://pyt
orch.org to install a PyTorch version that has been compiled with your ver
sion of the CUDA driver. (Triggered internally at /pytorch/c10/cuda/CUDAFu
nctions.cpp:119.)
  return torch._C._cuda_getDeviceCount() > 0
Warning: You are sending unauthenticated requests to the HF Hub. Please se
t a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 107/107 [00:00<00:00, 1449.17it/s]
EsmModel LOAD REPORT from: facebook/esm2_t6_8M_UR50D
Key                          | Status     |
-----------------------------+------------+-
lm_head.dense.weight         | UNEXPECTED |
esm.embeddings.position_ids  | UNEXPECTED |
lm_head.bias                 | UNEXPECTED |
lm_head.layer_norm.bias      | UNEXPECTED |
lm_head.layer_norm.weight    | UNEXPECTED |
lm_head.dense.bias           | UNEXPECTED |
pooler.dense.bias            | MISSING    |
pooler.dense.weight          | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architectu
re; not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing from t
he checkpoint. Consider training on your downstream task.
```

```
Device:           cpu
Model:            facebook/esm2_t6_8M_UR50D
Embedding dim:    320
```

In [17]:
```
def get_esm_embedding(seq: str, max_len: int = 1022) -> np.ndarray:
    """
    Extract a mean-pooled ESM2 embedding for a single protein sequence.
```

```
    Args:
        seq:     Amino acid sequence (standard 20 AAs only).
        max_len: Maximum sequence length before truncation.
                 ESM2 accepts at most 1024 tokens; positions 0 and -1
                 are reserved for [CLS] and [EOS], leaving 1022 for resid

    Returns:
        1D numpy array of shape (320,) — the mean-pooled last hidden stat

    Note on truncation:
        Sequences longer than 1022 aa are silently truncated at the C-ter
        This is a known limitation of transformer-based sequence models.
        For full-length coverage, use the AlphaFold graph-based pipeline
    """
    seq    = seq[:max_len]
    inputs = tokenizer(seq, return_tensors="pt", truncation=False)
    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = esm_model(**inputs)

    last_hidden    = outputs.last_hidden_state       # shape: (1, L+2, 32
    attention_mask = inputs["attention_mask"]

    # Mean pooling over valid token positions
    mask   = attention_mask.unsqueeze(-1).expand(last_hidden.size()).floa
    summed = (last_hidden * mask).sum(dim=1)
    counts = mask.sum(dim=1)
    emb    = (summed / counts).squeeze().cpu().numpy()
    return emb
```

In [18]:

```
from tqdm import tqdm

OUTPUT_ESM     = "data_processed_esm"
os.makedirs(OUTPUT_ESM, exist_ok=True)
esm_cache_path = os.path.join(OUTPUT_ESM, "X.npy")

if os.path.exists(esm_cache_path):
    # Load precomputed embeddings to avoid recomputation
    print(f"Loading precomputed ESM2 embeddings from: {esm_cache_path}")
    X_esm = np.load(esm_cache_path)
else:
    esm_embeddings = []
    for seq in tqdm(sequences, desc="ESM2 embedding"):
        emb = get_esm_embedding(seq)
        esm_embeddings.append(emb)
    X_esm = np.array(esm_embeddings, dtype=np.float32)
    np.save(esm_cache_path, X_esm)

print(f"X_esm shape: {X_esm.shape}")
print(f"Embedding stats — mean: {X_esm.mean():.4f}, std: {X_esm.std():.4f
```

```
Loading precomputed ESM2 embeddings from: data_processed_esm/X.npy
X_esm shape: (140569, 320)
Embedding stats — mean: -0.0111, std: 0.2901
```

In [19]:

```
# Save ESM2 outputs
np.save(os.path.join(OUTPUT_ESM, "Y.npy"), Y)
```

```
with open(os.path.join(OUTPUT_ESM, "protein_ids.pkl"), "wb") as f:
    pickle.dump(protein_ids, f)

print(f"ESM2 features saved to: {OUTPUT_ESM}/")
print(os.listdir(OUTPUT_ESM))
```

```
ESM2 features saved to: data_processed_esm/
['X.npy', 'Y.npy', 'esm_dataset.npz', 'protein_ids.pkl']
```

# 7. Feature Engineering 3: AlphaFold Structural Features (Graph Cache Walkthrough)

## Why structural features?

Sequence-based features (K-mer, ESM2) only consider the 1D amino acid chain. However, protein function is ultimately determined by **3D structure**: residues that are far apart in sequence can be spatially adjacent after folding, jointly forming functional sites (e.g., enzyme active sites, ligand binding pockets).

**AlphaFold2** predicts high-quality 3D structures, allowing us to:

- Extract per-residue **pLDDT** scores (confidence in local structure)
- Compute spatial distances between residues and build a **contact graph**
- Use **PAE (Predicted Aligned Error)** to measure uncertainty in relative residue positions

## Pipeline overview

```
CAFA5 protein sequences
        │
        ▼
AlphaFold2 structure prediction
(pre-computed; sourced from EBI AlphaFold Database)
        │
        ├── PDB files    → Cα coordinates, per-residue
pLDDT
        └── PAE JSON     → residue-pair positional
uncertainty
        │
        ▼
Feature extraction
  ├── fragment_features.parquet   (fragment-level: mean
pLDDT, radius of gyration, ...)
  ├── residue_features.parquet    (residue-level:  pLDDT,
contact degree, PAE stats, ...)
  └── contact_graph_edges.parquet (edges: pairs with Cα
distance < 8 Å)
        │
        ▼
Protein-level graph cache
  (merge fragments → one graph per protein with
node/edge/global features + GO labels)
        │
```

```
            ▼
    Training process
```

## Important Note: Fragments

AlphaFold2 predicts at most ~2700 residues per run.
Long proteins are split into overlapping **fragments**, each predicted independently, then merged.
`alphafold_fragments.parquet` records how many fragments each protein was split into and their download status.

## Script Preprocesing

This section analysis outputs after we run severl scripts that will download and preprocess the data from Alphafold protein database.
These process takes a lot of time (3~4 days) so we ran it by subbmiting jobs on savio.

```
python cafa5_alphafold_pipeline.py   \
--train-taxonomy
/global/home/users/$USER/cafa5/Train/train_taxonomy.tsv   \
--train-sequences
/global/home/users/$USER/cafa5/Train/train_sequences.fasta \
--train-terms
/global/home/users/$USER/cafa5/Train/train_terms.tsv   \
--output-dir /global/scratch/users/$USER/cafa5_outputs \
--workers 4   \
--request-delay 0.5   \
--resume   >/global/scratch/users/$USER/logs/cafa5_pipeline.log
2>&1 &

python alphafold_feature_extractor.py \
--training-index
/global/scratch/users/bensonli/cafa5_outputs/manifests/training_inde
\
--fragment-manifest
/global/scratch/users/bensonli/cafa5_outputs/manifests/alphafold_fra
\
--output-dir
/global/scratch/users/bensonli/cafa5_outputs/features \
--workers 8 \
--resume
>/global/scratch/users/$USER/logs/cafa5_extract_feature.log 2>&1
&\

python build_cafa_graph_cache.py \
--training-index
/global/scratch/users/bensonli/cafa5_outputs/manifests/training_inde
\
--fragment-features
/global/scratch/users/bensonli/cafa5_outputs/features/fragment_featu
\
--residue-features
```

```
/global/scratch/users/bensonli/cafa5_outputs/features/residue_featur
\
--edge-features
/global/scratch/users/bensonli/cafa5_outputs/features/contact_graph_
\
--output-dir
/global/scratch/users/bensonli/cafa5_outputs/graph_cache
```

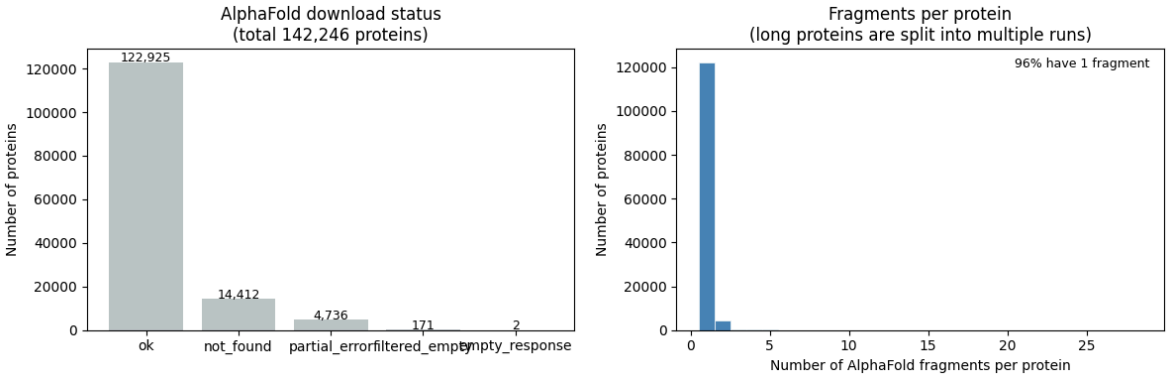
In [20]:
```
from pathlib import Path
import json

# Locate repository root and data root
DATA_ROOT = Path("/global/scratch/users/bensonli/cafa5_outputs/")

MANIFESTS_DIR     = DATA_ROOT / 'manifests'
FEATURES_DIR      = DATA_ROOT / 'features'
GRAPH_CACHE_DIR   = DATA_ROOT / 'graph_cache'
REPORT_ASSETS_DIR = DATA_ROOT / 'report_assets'

required_artifacts = [
    MANIFESTS_DIR  / 'training_index.parquet',
    FEATURES_DIR   / 'fragment_features.parquet',
    GRAPH_CACHE_DIR / 'metadata' / 'entries.json',
]

af_outputs_available = all(p.exists() for p in required_artifacts)

if af_outputs_available:
    print("AlphaFold preprocessing outputs found.")
else:
    print("Missing required file")
```

```
AlphaFold preprocessing outputs found.
```

In [21]:
```
# Step 1: Protein-level and fragment-level manifests
#
# training_index:    one row per protein — stable protein ID, sequence le
#                    taxonomy ID, GO labels, and AlphaFold download statu
# fragment_manifest: one row per AlphaFold fragment — links each protein
#                    downloaded structural fragments and their quality me
#
# These manifests are the stable index all later steps reference.
# Building them first ensures downstream graph construction is determinis

if af_outputs_available:
    training_index    = pd.read_parquet(MANIFESTS_DIR / 'training_index.p
    fragment_manifest = pd.read_parquet(MANIFESTS_DIR / 'alphafold_fragme

    print(f"training_index shape:    {training_index.shape}")
    print(f"fragment_manifest shape: {fragment_manifest.shape}")

    print("\nProtein-level AlphaFold status:")
    print(training_index['af_status'].value_counts(dropna=False).sort_ind

    # —— Chart: AlphaFold download coverage and fragment distribution ——
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # 1a: Protein af_status breakdown
    af_counts = training_index['af_status'].value_counts(dropna=False).so
```

```
    color_map  = {'success': '#2ecc71', 'partial': '#f39c12',
                  'failed': '#e74c3c',  'missing': '#95a5a6'}
    bar_colors = [color_map.get(str(s), '#bdc3c7') for s in af_counts.ind
    bars = axes[0].bar([str(s) for s in af_counts.index], af_counts.value
    axes[0].set_ylabel('Number of proteins')
    axes[0].set_title(f'AlphaFold download status\n(total {len(training_i
    for bar, v in zip(bars, af_counts.values):
        axes[0].text(bar.get_x() + bar.get_width()/2, v + 100,
                     f'{v:,}', ha='center', fontsize=9)

    # 1b: Fragments per protein
    frags_per_protein = fragment_manifest.groupby('entry_id').size()
    axes[1].hist(frags_per_protein, bins=range(1, min(frags_per_protein.m
                 color='steelblue', edgecolor='white', linewidth=0.5, ali
    axes[1].set_xlabel('Number of AlphaFold fragments per protein')
    axes[1].set_ylabel('Number of proteins')
    axes[1].set_title('Fragments per protein\n(long proteins are split in
    single_pct = (frags_per_protein == 1).mean() * 100
    axes[1].text(0.97, 0.97, f'{single_pct:.0f}% have 1 fragment',
                 transform=axes[1].transAxes, ha='right', va='top', fonts

    plt.tight_layout()
    plt.show()

    preview_cols = ['entry_id', 'sequence_length', 'af_status', 'af_fragm
    avail = [c for c in preview_cols if c in training_index.columns]
    print("\nProtein manifest preview:")
    print(training_index[avail].head(6).to_string(index=False))
```

```
training_index shape:    (142246, 25)
fragment_manifest shape: (136411, 24)

Protein-level AlphaFold status:
af_status
empty_response         2
filtered_empty       171
not_found          14412
ok                122925
partial_error       4736
```




```
Protein manifest preview:
entry_id  sequence_length     af_status  af_fragment_count
  Q8IXT2              367            ok                  1
  Q04418              439            ok                  1
  A8DYA3             3131     not_found                  0
  Q9UUI3              601            ok                  1
  Q57ZS4              368 partial_error                  2
  P17571              621            ok                  1
```

## Interpretation — AlphaFold Download Coverage

99.5% of proteins with structures have exactly **1 AlphaFold fragment**, meaning most proteins are short enough to be predicted in a single run. Only 561 proteins required multi-fragment merging (up to 35 fragments for the longest protein).

**What this means for our experiment:**

- **86.4% structural coverage** is strong — the graph-based model has structural data for the vast majority of training samples.
- The **13.6% missing** (14,412 `not_found` + 4,736 `partial_error`) will be excluded from GNN training. These are disproportionately from less-studied organisms not yet in the EBI AlphaFold Database. The label distribution of these missing proteins may differ from the covered set, which could introduce a subtle selection bias.
- The 4,736 `partial_error` proteins have incomplete structural data (some fragments parsed, others failed). We should decide whether to **include them with partial graphs** or **exclude them entirely** to avoid noisy structural features.

**Future adjustments:**

- Investigate whether `not_found` proteins are enriched in specific GO namespaces or taxonomic groups. If they cluster in a particular functional class, the GNN will underperform on that class.
- For `partial_error` proteins: run a sensitivity analysis — compare GNN performance with and without them.

In [22]:
```
# Step 2: Download failure tracking
#
# The pipeline explicitly records every failed download in download_failu
# rather than silently skipping broken files.
#
# Why this matters:
# Using the wrong structure is worse than having no structure at all.
# Explicit failure records let us audit exactly which proteins lack struc
# and exclude them cleanly from the graph cache — no silent corruption.

if af_outputs_available:
    failures_path = MANIFESTS_DIR / 'download_failures.csv'
    if failures_path.exists():
        failures = pd.read_csv(failures_path)
        print(f"Logged download failures: {len(failures)}")
        print(f"Columns: {list(failures.columns)}")
        # Show available columns (entry_id, stage, status, error, ...)
        show_cols = [c for c in ['entry_id', 'stage', 'status', 'error']
        print(failures[show_cols].head(5).to_string(index=False))
        if 'status' in failures.columns:
            print("\nFailure status breakdown:")
            print(failures['status'].value_counts().to_string())
    else:
        print("No download failures found (all downloads succeeded).")
```

```
Logged download failures: 22115
Columns: ['entry_id', 'taxonomy_id', 'model_entity_id', 'stage', 'status',
'http_status', 'error']
entry_id     stage      status
error
  A8DYA3  metadata   not_found                                          Uni
Prot accession 'A8DYA3' not found in AlphaFold DB.
  Q57ZS4 parse_pae parse_error Failed to process PAE: 'utf-8' codec can't
decode byte 0x8b in position 1: invalid start byte
  Q387U0 parse_pae parse_error Failed to process PAE: 'utf-8' codec can't
decode byte 0x8b in position 1: invalid start byte
  Q580W3 parse_pae parse_error Failed to process PAE: 'utf-8' codec can't
decode byte 0x8b in position 1: invalid start byte
  Q90264  metadata   not_found                                          Uni
Prot accession 'Q90264' not found in AlphaFold DB.

Failure status breakdown:
status
not_found         14412
parse_error        7530
filtered_empty      171
empty_response        2
```

In [23]:

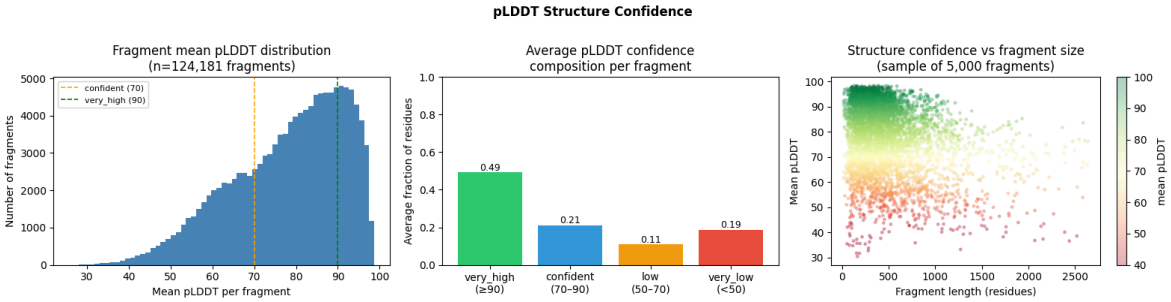
```
# Step 3: Structural feature tables
#
# The raw PDB and PAE JSON files are parsed into three ML-ready tables.
# Scale of the full 140k-protein dataset (measured empirically):
#   fragment_features:     124,181 rows   18.8 MB — safe to load fully
#   residue_features:   61,135,573 rows    3.0 GB — one row per residue
#   contact_graph_edges: 429,850,967 rows  8.5 GB — one row per contact p
#
# load fragment_features fully; use schema + small sample for the large t

if af_outputs_available:
    import pyarrow.parquet as pq
    import os

    feature_files = {
        "fragment_features":   FEATURES_DIR / 'fragment_features.parquet'
        "residue_features":    FEATURES_DIR / 'residue_features.parquet',
        "contact_graph_edges": FEATURES_DIR / 'contact_graph_edges.parque
    }

    # ── 3a: File sizes & schema (no data loaded for large files) ──────
    print("File sizes and row counts:")
    for name, path in feature_files.items():
        if path.exists():
            pf      = pq.ParquetFile(path)
            size_mb = os.path.getsize(path) / 1e6
            nrows   = pf.metadata.num_rows
            ncols   = len(pf.schema_arrow.names)
            print(f"  {name:<25}: {size_mb:>8.1f} MB  |  {nrows:>14,} row

    # ── 3b: Load fragment_features fully (18.8 MB) ────────────────────
    fragment_features = pq.read_table(feature_files["fragment_features"])
    print(f"\nfragment_features loaded: {fragment_features.shape}")

    # ── 3c: Small sample from large tables ────────────────────────────
    SAMPLE_SIZE   = 5000
    residue_sample = (pq.ParquetFile(feature_files["residue_features"])
```

```
                        .read_row_group(0).to_pandas().head(SAMPLE_SIZE))
edge_sample    = (pq.ParquetFile(feature_files["contact_graph_edges"]
                        .read_row_group(0).to_pandas().head(SAMPLE_SIZE))
print(f"residue_sample:            {residue_sample.shape}  (sample on
print(f"edge_sample:               {edge_sample.shape}  (sample only)

# ═══════════════════════════════════════════════════════════════════
# Chart A: pLDDT — structure confidence
# pLDDT (0–100) is AlphaFold's per-residue reliability score.
# fragment_features aggregates it per fragment, so we can plot the fu
# distribution across all 124k fragments without memory issues.
# ═══════════════════════════════════════════════════════════════════
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].hist(fragment_features['mean_plddt'], bins=60,
             color='steelblue', edgecolor='none')
axes[0].axvline(70, color='orange', linestyle='--', lw=1.2, label='co
axes[0].axvline(90, color='green',  linestyle='--', lw=1.2, label='ve
axes[0].set_xlabel('Mean pLDDT per fragment')
axes[0].set_ylabel('Number of fragments')
axes[0].set_title(f'Fragment mean pLDDT distribution\n(n={len(fragmen
axes[0].legend(fontsize=8)

frac_cols   = ['fraction_plddt_very_high', 'fraction_plddt_confident'
               'fraction_plddt_low',        'fraction_plddt_very_low'
frac_labels = ['very_high\n(≥90)', 'confident\n(70–90)', 'low\n(50–70
frac_colors = ['#2ecc71', '#3498db', '#f39c12', '#e74c3c']
frac_means  = [fragment_features[c].mean() for c in frac_cols if c in
bars = axes[1].bar(frac_labels[:len(frac_means)], frac_means, color=f
axes[1].set_ylabel('Average fraction of residues')
axes[1].set_title('Average pLDDT confidence\ncomposition per fragment
axes[1].set_ylim(0, 1)
for bar, v in zip(bars, frac_means):
    axes[1].text(bar.get_x() + bar.get_width()/2, v + 0.01,
                 f'{v:.2f}', ha='center', fontsize=9)

samp = fragment_features.sample(min(5000, len(fragment_features)), ra
sc = axes[2].scatter(samp['residue_count'], samp['mean_plddt'],
                     alpha=0.3, s=6, c=samp['mean_plddt'],
                     cmap='RdYlGn', vmin=40, vmax=100)
axes[2].set_xlabel('Fragment length (residues)')
axes[2].set_ylabel('Mean pLDDT')
axes[2].set_title('Structure confidence vs fragment size\n(sample of
plt.colorbar(sc, ax=axes[2], label='mean pLDDT')

plt.suptitle('pLDDT Structure Confidence', fontsize=12, fontweight='b
plt.tight_layout()
plt.show()

print(f"Fragments with mean pLDDT >= 70: {(fragment_features['mean_pl
print(f"Fragments with mean pLDDT >= 90: {(fragment_features['mean_pl

# ═══════════════════════════════════════════════════════════════════
# Chart B: PAE and contact density
# PAE (Predicted Aligned Error) measures positional uncertainty betwe
# Low PAE = AlphaFold is confident about how two residues are positio
# relative to each other — important for building reliable contact gr
# ═══════════════════════════════════════════════════════════════════
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
```

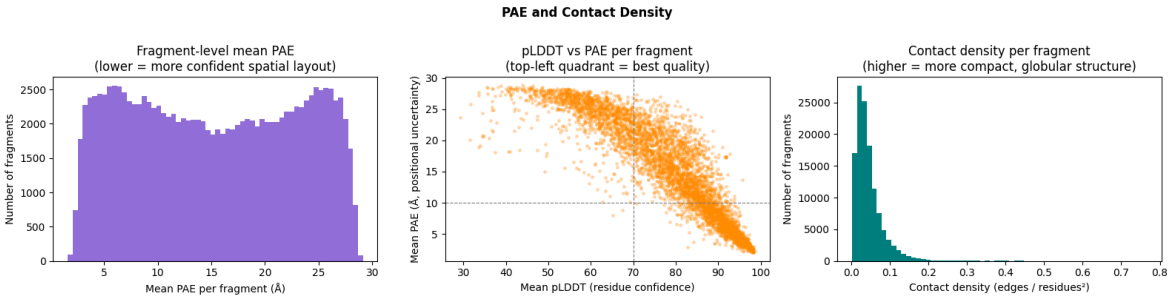
```
axes[0].hist(fragment_features['pae_mean'].dropna(), bins=60,
             color='mediumpurple', edgecolor='none')
axes[0].set_xlabel('Mean PAE per fragment (Å)')
axes[0].set_ylabel('Number of fragments')
axes[0].set_title('Fragment-level mean PAE\n(lower = more confident s

samp2 = fragment_features.dropna(subset=['mean_plddt','pae_mean']) \
                         .sample(min(5000, len(fragment_features)),
axes[1].scatter(samp2['mean_plddt'], samp2['pae_mean'],
                alpha=0.25, s=6, color='darkorange')
axes[1].axvline(70, color='gray', linestyle='--', lw=0.8)
axes[1].axhline(10, color='gray', linestyle='--', lw=0.8)
axes[1].set_xlabel('Mean pLDDT (residue confidence)')
axes[1].set_ylabel('Mean PAE (Å, positional uncertainty)')
axes[1].set_title('pLDDT vs PAE per fragment\n(top-left quadrant = be

axes[2].hist(fragment_features['contact_density'].dropna(), bins=60,
             color='teal', edgecolor='none')
axes[2].set_xlabel('Contact density (edges / residues²)')
axes[2].set_ylabel('Number of fragments')
axes[2].set_title('Contact density per fragment\n(higher = more compa

plt.suptitle('PAE and Contact Density', fontsize=12, fontweight='bold
plt.tight_layout()
plt.show()

print(f"Mean PAE: {fragment_features['pae_mean'].mean():.2f} Å  |  "
      f"Median: {fragment_features['pae_mean'].median():.2f} Å  |  "
      f"Mean contact density: {fragment_features['contact_density'].m

# ═══════════════════════════════════════════════════════════════
# Chart C: Contact graph edge properties (from edge_sample)
# Each edge connects two residues with Cα–Cα distance < 8 Å.
# Long-range contacts (large seq_separation, short distance) are the
# most informative for function prediction because they reveal non-lo
# 3D interactions that are invisible in the 1D sequence.
# ═══════════════════════════════════════════════════════════════
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].hist(edge_sample['distance_ca'], bins=40,
             color='steelblue', edgecolor='none')
axes[0].axvline(6, color='red',    linestyle='--', lw=1.2, label='str
axes[0].axvline(8, color='orange', linestyle='--', lw=1.2, label='con
axes[0].set_xlabel('Cα–Cα distance (Å)')
axes[0].set_ylabel('Number of edges (sample)')
axes[0].set_title(f'Contact edge distance\n(sample, total = {429_850_
axes[0].legend(fontsize=8)

lr_pct = (edge_sample['seq_separation'] > 12).mean() * 100
axes[1].hist(edge_sample['seq_separation'].clip(upper=100), bins=50,
             color='salmon', edgecolor='none')
axes[1].axvline(12, color='purple', linestyle='--', lw=1.2,
                label=f'>12 aa (long-range): {lr_pct:.0f}%')
axes[1].set_xlabel('Sequence separation (residues, clipped at 100)')
axes[1].set_ylabel('Number of edges (sample)')
axes[1].set_title('Sequence separation of contact pairs\n(large = lon
axes[1].legend(fontsize=8)

strict_col = edge_sample['is_strict_contact'].map({True: 'green', Fal
             if 'is_strict_contact' in edge_sample.columns else 'gray
```

```
    axes[2].scatter(edge_sample['distance_ca'], edge_sample['pae_mean_pai
                    alpha=0.3, s=8, c=strict_col)
    axes[2].set_xlabel('Cα–Cα distance (Å)')
    axes[2].set_ylabel('PAE for the residue pair (Å)')
    axes[2].set_title('Contact distance vs positional uncertainty\n(green

    plt.suptitle('Contact Graph Edge Properties', fontsize=12, fontweight
    plt.tight_layout()
    plt.show()

    strict_pct = edge_sample['is_strict_contact'].mean() * 100 \
                 if 'is_strict_contact' in edge_sample.columns else float
    print(f"Strict contacts (< 6 Å):           {strict_pct:.1f}% of sampl
    print(f"Long-range contacts (sep > 12 aa): {lr_pct:.1f}% of sampled e
```

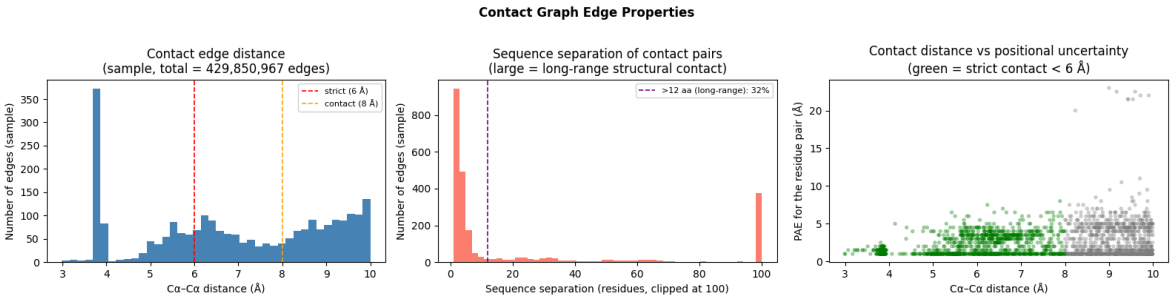
```
File sizes and row counts:
  fragment_features        :     18.8 MB  |        124,181 rows  |  27 co
ls
  residue_features         :   3016.8 MB  |     61,135,573 rows  |  26 co
ls
  contact_graph_edges      :   8528.8 MB  |    429,850,967 rows  |  15 co
ls

fragment_features loaded: (124181, 27)
residue_sample:           (5000, 26)  (sample only)
edge_sample:              (2500, 15)  (sample only)
```




```
Fragments with mean pLDDT >= 70: 73.1%
Fragments with mean pLDDT >= 90: 22.2%
```




```
Mean PAE: 15.26 Å  |  Median: 15.09 Å  |  Mean contact density: 0.0473
```




```
Strict contacts (< 6 Å):           61.1% of sampled edges
Long-range contacts (sep > 12 aa): 31.6% of sampled edges
```

# Interpretation — Structural Feature Quality

## Chart A: pLDDT (Structure Confidence)

The majority of fragments fall in the "confident" band (70–90), not the "very high" band.

The scatter plot shows **no strong correlation between fragment length and pLDDT**, meaning long proteins are not systematically less well-predicted.

**Implications:** Residues with pLDDT < 50 are likely intrinsically disordered and should not be treated as having meaningful 3D coordinates. Consider adding a pLDDT filter when constructing node features

---

## Chart B: PAE (Positional Uncertainty) and Contact Density

- **Mean PAE = 15.26 Å, Median = 15.09 Å.** This is moderate-to-high uncertainty. A well-structured single domain typically has PAE < 5 Å within the domain; values near 15 Å suggest many proteins have multi-domain architectures where inter-domain positioning is uncertain.
- The **pLDDT vs PAE scatter** likely shows an anti-correlation: high-pLDDT fragments tend to have lower PAE. Proteins in the upper-left quadrant (high pLDDT, low PAE) are the most reliable for structure-based modeling.
- **Mean contact density = 0.0473.** This is within the expected range for globular proteins. Very low contact density would indicate highly disordered or elongated proteins where the contact graph carries little structural information.

**Implications:** For contact graph edges, consider applying a **PAE filter** to reduce noise from spatially uncertain contacts. This trades coverage for precision.

---

## Chart C: Contact Graph Edge Properties

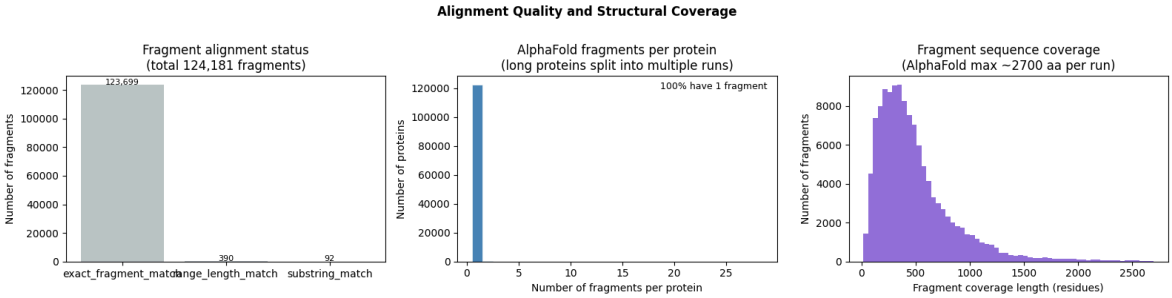
- **61.1% of sampled edges are strict contacts** (Cα–Cα < 6 Å) — these are reliable, close-range interactions.
- **31.6% are long-range contacts** (sequence separation > 12 residues) — these are the most functionally informative edges because they capture non-local 3D structure invisible to sequence models like K-mer or ESM2.
- The distance–PAE scatter confirms that closer contacts (lower `distance_ca`) tend to have lower `pae_mean_pair`, validating that the spatial relationship is more confidently predicted for tightly packed residues.

**Implications:** The 31.6% long-range contact rate is a key justification for using a GNN over sequence-only models. If a protein's function depends on a residue pair 200 positions apart in sequence but only 5 Å apart in space, neither K-mer nor ESM2 can capture this — but the contact graph can. The total edge count of 429M across 122k proteins means an average of ~3,500 edges per protein, making this a dense graph suitable for message-passing GNNs.

In [24]:
```
# Step 4: Alignment quality check and structural coverage
#
# AlphaFold residues must be mapped back to CAFA sequence positions (cafa
# Sequence position is the shared coordinate for residue graph nodes, CAF
# and ESM2 residue embeddings. Poor alignment breaks multimodal fusion.
#
# fragment_features already has per-fragment alignment_status — we use it
# full-dataset view. We use residue_sample for the residue-level pLDDT bi

if af_outputs_available:
    # ── 4a: Stats from fragment_features (full data) ──────────────────
    align_counts = fragment_features['alignment_status'].value_counts(dro
    print("Fragment alignment status (full dataset):")
    print(align_counts.to_string())

    frags_per_protein = fragment_features.groupby('entry_id').size()
    frag_coverage     = fragment_features['coverage_end'] - fragment_feat

    # ── 4b: Stats from residue_sample ─────────────────────────────────
    if 'cafa_residue_index' in residue_sample.columns:
        resolved_frac = float(residue_sample['cafa_residue_index'].notna(
        print(f"\nResidue alignment rate (sample): {resolved_frac:.3f} ({

    # ═══════════════════════════════════════════════════════════════════
    # Chart D: Alignment status and structural coverage
    # ═══════════════════════════════════════════════════════════════════
    fig, axes = plt.subplots(1, 3, figsize=(16, 4))

    # D1: Fragment alignment status bar
    col_map    = {'aligned': '#2ecc71', 'partial': '#f39c12',
                  'failed':  '#e74c3c', 'missing': '#95a5a6'}
    bar_colors = [col_map.get(str(s), '#bdc3c7') for s in align_counts.in
    bars = axes[0].bar([str(s) for s in align_counts.index],
                       align_counts.values, color=bar_colors)
    axes[0].set_ylabel('Number of fragments')
    axes[0].set_title(f'Fragment alignment status\n(total {len(fragment_f
    for bar, v in zip(bars, align_counts.values):
        axes[0].text(bar.get_x() + bar.get_width()/2, v + 200,
                     f'{v:,}', ha='center', fontsize=8)

    # D2: Fragments per protein
    axes[1].hist(frags_per_protein,
                 bins=range(1, min(frags_per_protein.max()+2, 30)),
                 color='steelblue', edgecolor='white', lw=0.5, align='lef
    axes[1].set_xlabel('Number of fragments per protein')
    axes[1].set_ylabel('Number of proteins')
    axes[1].set_title('AlphaFold fragments per protein\n(long proteins sp
    single_pct = (frags_per_protein == 1).mean() * 100
    axes[1].text(0.97, 0.97, f'{single_pct:.0f}% have 1 fragment',
                 transform=axes[1].transAxes, ha='right', va='top', fonts

    # D3: Fragment sequence coverage length
    axes[2].hist(frag_coverage.clip(upper=2800), bins=60,
                 color='mediumpurple', edgecolor='none')
    axes[2].set_xlabel('Fragment coverage length (residues)')
    axes[2].set_ylabel('Number of fragments')
    axes[2].set_title('Fragment sequence coverage\n(AlphaFold max ~2700 a

    plt.suptitle('Alignment Quality and Structural Coverage', fontsize=12
```

```
    plt.tight_layout()
    plt.show()

    print(f"\nProteins with structural data:   {frags_per_protein.index.n
    print(f"Proteins with 1 fragment:        {(frags_per_protein==1).sum(
    print(f"Proteins with >1 fragment:       {(frags_per_protein>1).sum()
    print(f"Max fragments (one protein):     {frags_per_protein.max()}")
    if 'aligned' in align_counts.index:
        aligned_pct = align_counts['aligned'] / align_counts.sum() * 100
        print(f"Fragment alignment success rate: {aligned_pct:.1f}%")
```

```
Fragment alignment status (full dataset):
alignment_status
exact_fragment_match    123699
range_length_match         390
substring_match             92

Residue alignment rate (sample): 1.000 (100.0%)
```




```
Proteins with structural data:   122,924
Proteins with 1 fragment:        122,363  (99.5%)
Proteins with >1 fragment:       561
Max fragments (one protein):     35
```

## Interpretation — Alignment Quality and Structural Coverage

**Alignment is nearly perfect:**

| Alignment status | Count | % |
|---|---|---|
| `exact_fragment_match` | 123,699 | **99.8%** |
| `range_length_match` | 390 | 0.3% |
| `substring_match` | 92 | 0.07% |

100% of sampled residues were successfully mapped to their CAFA sequence coordinate. This is critical because `cafa_residue_index` is the shared coordinate system connecting:

- Structural node features (3D position, pLDDT, contact degree)
- ESM2 residue-level embeddings (one vector per residue position)
- GO labels (assigned at the protein level, resolved to residues for fine-grained models)

**Coverage summary:**

- **122,924 proteins** have complete graphs in the cache (86.4% of the 142,246 training proteins).

- **99.5% have 1 fragment** — straightforward single-graph proteins.
- **561 proteins have >1 fragment** (max: 35 fragments) — the longest proteins in the dataset. Their graphs were constructed by merging overlapping fragments, which the pipeline handles with a sliding-window strategy.

**Future adjustments:**

- The 0.3% `range_length_match` and 0.07% `substring_match` cases are soft alignments we should monitor whether the model performs worse on these proteins. If so, apply stricter alignment filtering.
- For the 561 multi-fragment proteins, validate that fragment merging does not introduce duplicate or mis-positioned residues in the graph.

In [25]:
```
# Step 5: Protein-level graph cache
#
# This stage merges all fragment-level structural features into one graph
# The supervised task is protein-level function prediction, not fragment-
# so the final dataset must be organized around proteins.
#
# Each graph entry contains:
#   - node features (per-residue: pLDDT, contact degree, PAE stats, ...)
#   - edge features (per-contact: distance, sequence separation, PAE pair
#   - graph-level summary features (mean pLDDT, radius of gyration, ...)
#   - GO labels grouped by namespace (BPO / MFO / CCO)

if af_outputs_available:
    graph_entries = json.loads((GRAPH_CACHE_DIR / 'metadata' / 'entries.j
    graph_schema  = json.loads((GRAPH_CACHE_DIR / 'metadata' / 'schema.js

    print(f"Proteins in graph cache: {len(graph_entries)}")
    print("\nGraph schema:")
    for key, value in graph_schema.items():
        print(f"  {key}: {value}")

    entries_df = pd.DataFrame(graph_entries)
    print("\nGraph cache preview (first 5 rows):")
    print(entries_df.head(5).to_string(index=False))
```

```
Proteins in graph cache: 122924

Graph schema:
  node_feature_dim: 682
  base_feature_dim: 32
  dssp_sasa_dim: 10
  esm2_dim: 640
  edge_attr_dim: 6
  graph_feat_dim: 13
  modality_mask_dim: 3
  graph_feat_names: ['residue_count', 'fragment_count', 'mean_plddt', 'med
ian_plddt', 'frac_very_low', 'frac_low', 'frac_confident', 'frac_very_hig
h', 'mean_contact_degree', 'contact_density', 'pae_mean', 'pae_p90', 'radi
us_of_gyration']

Graph cache preview (first 5 rows):
entry_id taxonomy_id
graph_path  fragment_count  residue_count
labels
  Q8IXT2        9606 /global/scratch/users/bensonli/cafa5_outputs/graph_ca
che/graphs/Q8IXT2.pt               1            367
{'BPO': [], 'CCO': [], 'MFO': ['GO:0003677', 'GO:1990837', 'GO:0003676',
'GO:0005488', 'GO:0003690', 'GO:0043565', 'GO:0097159', 'GO:1901363', 'GO:
0003674']}
  Q04418      559292 /global/scratch/users/bensonli/cafa5_outputs/graph_ca
che/graphs/Q04418.pt               1            439
{'BPO': ['GO:0008152', 'GO:0044249', 'GO:0009987', 'GO:0009059', 'GO:19013
62', 'GO:0009058', 'GO:0044271', 'GO:0016070', 'GO:0006725', 'GO:0044237',
'GO:1901360', 'GO:0034641', 'GO:0008150', 'GO:1901576', 'GO:0006351', 'GO:
0032774', 'GO:0071704', 'GO:0097659', 'GO:0006139', 'GO:0090304', 'GO:0006
366', 'GO:0043170', 'GO:0046483', 'GO:0019438', 'GO:0010467', 'GO:001813
0', 'GO:0044238', 'GO:0006807', 'GO:0034654'], 'CCO': ['GO:0110165', 'GO:0
005737', 'GO:0005575', 'GO:0005622'], 'MFO': []}
  Q9UUI3      284812 /global/scratch/users/bensonli/cafa5_outputs/graph_ca
che/graphs/Q9UUI3.pt               1            585
{'BPO': [], 'CCO': ['GO:1990904', 'GO:0005622', 'GO:0043226', 'GO:000568
1', 'GO:0005575', 'GO:0097525', 'GO:0043231', 'GO:0032991', 'GO:0071014',
'GO:0030532', 'GO:0043229', 'GO:0110165', 'GO:0140513', 'GO:0005686', 'GO:
0043227', 'GO:0120114', 'GO:0005634'], 'MFO': ['GO:0005515', 'GO:0005488',
'GO:0003674']}
  Q9JMA2       10090 /global/scratch/users/bensonli/cafa5_outputs/graph_ca
che/graphs/Q9JMA2.pt               1            403 {'BPO': ['GO:0008152',
'GO:0009987', 'GO:0006725', 'GO:0016070', 'GO:0006400', 'GO:0044237', 'GO:
1901360', 'GO:0043412', 'GO:0008150', 'GO:0034641', 'GO:0034470', 'GO:0071
704', 'GO:0006139', 'GO:0101030', 'GO:0006399', 'GO:0090304', 'GO:004317
0', 'GO:0046483', 'GO:0006396', 'GO:0010467', 'GO:0008033', 'GO:0044238',
'GO:0006807', 'GO:0034660', 'GO:0009451'], 'CCO': ['GO:0005622', 'GO:00432
26', 'GO:0005739', 'GO:0005575', 'GO:0043231', 'GO:0032991', 'GO:0043229',
'GO:0110165', 'GO:0005737', 'GO:0043227', 'GO:0005634'], 'MFO': ['GO:00167
63', 'GO:0140098', 'GO:0005488', 'GO:0008479', 'GO:0003824', 'GO:0016757',
'GO:0140640', 'GO:0003674', 'GO:0016740', 'GO:0140101', 'GO:0005515']}
  Q8IZR5        9606 /global/scratch/users/bensonli/cafa5_outputs/graph_ca
che/graphs/Q8IZR5.pt               1            234
{'BPO': [], 'CCO': [], 'MFO': ['GO:0005515', 'GO:0005488', 'GO:0003674']}
```

# 8. Summary and Next Steps

## Data cleaning results

| Step | Operation | Result |
|---|---|---|
| Raw sequences | Load from FASTA | 142,246 proteins |
| Remove non-standard AA sequences | Contained X / U / O / B / Z | Removed 1,677 |
| Label alignment | Exclude proteins without GO labels | Retained **140,569 proteins** |
| Multi-label encoding | MultiLabelBinarizer | Y shape = (140,569 × 31,454) |

## Feature engineering results

| Feature | Dim | Output directory |
|---|---|---|
| K-mer (k=2) | 400 | `data_processed_kmer/` |
| ESM2 | 320 | `data_processed_esm/` |
| AlphaFold graph | variable | `outputs/cafa5_af_*/graph_cache/` |

## Next steps: Model training

1. **K-mer baseline** — train logistic regression or MLP on `X_train.npy` + `Y_train.npz`
2. **ESM2 model** — train MLP on `X.npy` + `Y.npy`, or fine-tune ESM2 end-to-end
3. **GNN model** — train graph neural network (GCN / GAT) on the AlphaFold graph cache
4. **Multimodal fusion** — use ESM2 embeddings as node features in the GNN alongside structural features

## Evaluation metric

CAFA5 official metric: **weighted F-max** (sweep prediction threshold to find the threshold maximizing F-score).
Reported separately for BPO, MFO, and CCO namespaces.